# Data Handling and Infrastructure for AI

## MLOps

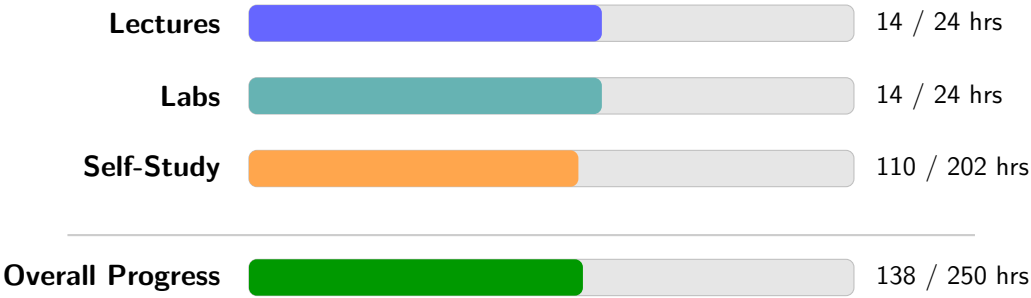
Tamás Grósz

SETU

2026



# Workload


Lectures
14 / 24 hrs
Labs
14 / 24 hrs
Self-Study
110 / 202 hrs
Overall Progress
138 / 250 hrs

# Learning Objectives

By the end of this lecture, you should be able to:

- Explain why machine learning requires more than a trained model.
- Describe the purpose of MLOps.
- Explain experiment tracking and model versioning.
- Understand the role of MLflow.
- Explain why containers are useful for ML systems.
- Understand the role of Docker in packaging ML applications.
- Explain the basic concepts of Kubernetes.
- Understand how Kubernetes supports scalable ML workloads.
- Describe the role of OpenShift as an enterprise Kubernetes platform.
- Connect ML pipelines, containers and orchestration into a production workflow.

# From ML Model to ML System

$$\text{Data} \rightarrow \text{Training} \rightarrow \text{Model}$$

But a real ML application also needs:

- data pipelines,
- training infrastructure,
- experiment tracking,
- model storage,
- deployment,
- monitoring,
- retraining,
- versioning,
- security and access control.

**Key idea**

A machine learning model is only one component of a larger software system.

# What Is MLOps?

**MLOps** is the set of practices and technologies used to develop, deploy, operate and maintain machine learning systems.

| **Machine Learning** | **Software Engineering** | **Operations** |
|---|---|---|
| Models | Git | Deployment |
| Training | Testing | Monitoring |
| Evaluation | CI/CD | Scaling |
| Data | Code review | Reliability |

$$\boxed{\text{MLOps} = \text{ML} + \text{Software Engineering} + \text{Operations}}$$

# Why Do We Need MLOps?

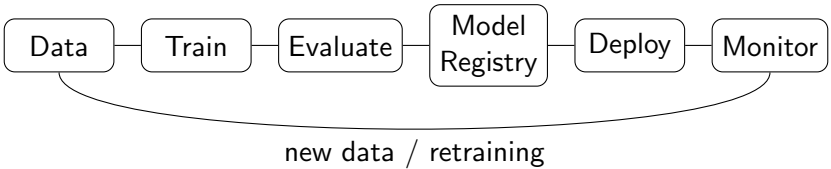
A model that works in a notebook is not necessarily a production system.
Consider:

- Which dataset was used?
- Which preprocessing pipeline?
- Which model version?
- Which hyperparameters?
- Which Python dependencies?
- Which hardware?
- Can another person reproduce the result?
- How do we deploy the model?
- How do we monitor it?
- What happens when the data changes?

**MLOps turns experiments into repeatable systems.**

# The ML Lifecycle

A production ML lifecycle might look like:




**MLOps principle**

The lifecycle is iterative rather than a one-time "train and deploy" process.

# The Reproducibility Problem

Imagine an experiment:

$$\texttt{experiment_42}$$

It achieved:

$$\text{Validation Accuracy} = 94.7\%$$

But what produced that number?

- Dataset version?
- Code commit?
- Learning rate?
- Batch size?
- Number of epochs?
- Model architecture?
- Random seed?
- Python/library versions?

# Experiment Tracking

An experiment tracking system records the details of training runs:

- parameters,
- hyperparameters,
- metrics,
- training time,
- dataset version,
- model artifacts,
- source code version,
- environment information.

It enables experiments to be compared systematically.

# MLflow

**MLflow** is an open-source platform for managing machine learning workflows.
It provides tools for:

- experiment tracking,
- model packaging,
- model registry,
- model lifecycle management.

A simplified MLflow workflow:

$$\boxed{\text{Training Code} \rightarrow \text{MLflow Tracking} \rightarrow \text{Metrics} + \text{Parameters} + \text{Artifacts}}$$

The goal is to make experiments:

**trackable, comparable and reproducible**

# MLflow Tracking

During training, an application can record:

$$\begin{aligned} \text{learning rate} &= 0.001 \\ \text{batch size} &= 64 \\ \text{epochs} &= 50 \\ \text{validation accuracy} &= 0.941 \\ \text{validation loss} &= 0.183 \end{aligned}$$

It can also store artifacts such as:

- trained model,
- confusion matrix, plots, evaluation reports.
- configuration files,

**Important**

MLflow does not replace your ML model or training framework. It provides infrastructure for managing the ML lifecycle.

# Model Registry

After training many models, we need to manage them.
A model registry can track:

$$\text{Model} \rightarrow \text{Version} \rightarrow \text{Stage / Status}$$

For example:




This creates a controlled path from:

$$\boxed{\text{Experiment} \rightarrow \text{Candidate} \rightarrow \text{Production Model}}$$

# Why Do We Need Containers?

Suppose the model works on the developer's machine.
It depends on:

- Python 3.x
- PyTorch
- NumPy
- specific library versions
- system libraries
- model files
- configuration

Another machine may have different versions.

"Works on my machine"

is not enough for production.

We want to package the application and its environment together.

# Docker

**Docker** is a widely used technology for building and running containers.
A Docker image contains the application environment needed to run the software.
Conceptually:

$$\text{Dockerfile} \rightarrow \text{Docker Image} \rightarrow \text{Container}$$

For an ML inference service, the image might contain:

- Python,
- ML libraries,
- application code,
- model files,
- configuration.

The same image can then be used in:

$$\text{Development} \rightarrow \text{Testing} \rightarrow \text{Production}$$

# Docker Image vs Container

These concepts are often confused.

**Image**

- Packaged application environment.
- Immutable template.
- Can be stored in a registry.
- Used to create containers.

**Container**

- Running instance of an image.
- Has its own runtime environment.
- Can be started, stopped and replaced.

$$\boxed{\text{Image} \xrightarrow{\text{run}} \text{Container}}$$

A useful analogy:

$$\text{Image} \approx \text{Class} \qquad \text{Container} \approx \text{Object}$$

# A Simple ML Container

Imagine an inference application:

```
POST /predict
```

The container includes:

- Inference API
- Trained Model
- ML Libraries
- Python Runtime

The Docker image provides a reproducible package containing these components.

**Benefit**

The deployment environment becomes part of the versioned artifact.

# Containers Are Not Enough

Suppose we have one container running our model.
What happens if:

- 10,000 users send requests?
- the container crashes?
- we need 10 replicas?
- a new model version is deployed?
- a machine fails?
- traffic suddenly increases?

We need a system that can manage many containers.

**This is where container orchestration comes in.**

# Kubernetes

**Kubernetes** is a platform for orchestrating containers. It can manage:

- container deployment,
- scaling,
- networking,
- service discovery,
- health checks,
- rolling updates,
- resource allocation.

Instead of manually running:

Container 1, Container 2, Container 3

we describe the desired state:

I want 3 healthy instances of this application

Kubernetes attempts to maintain that state.

# Kubernetes: Desired State

A key Kubernetes idea is:

$$\boxed{\text{Desired State} \rightarrow \text{Control Loop} \rightarrow \text{Actual State}}$$

For example:

$$\text{Desired replicas} = 3$$

If one container crashes:

$$3 \rightarrow 2$$

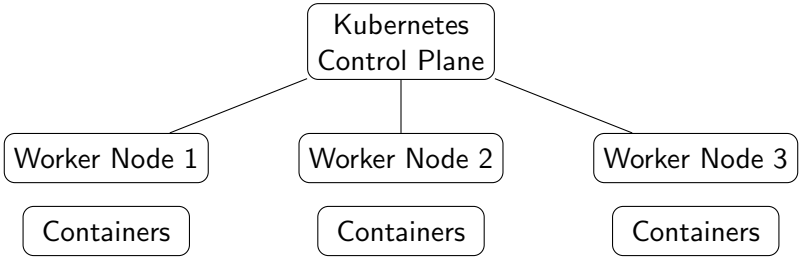
Kubernetes detects the difference and creates another:

$$2 \rightarrow 3$$

**Key idea**

Kubernetes continuously works to make the running system match the declared configuration.

# Kubernetes Architecture — Simplified




The control plane manages the desired state while worker nodes run the workloads.

# Kubernetes and ML Inference

Suppose we deploy an ML inference service.

$$\text{Client} \rightarrow \text{Service} \rightarrow \text{ML Pods}$$

We might have:

Pod 1 Pod 2 Pod 3

Kubernetes can:

- distribute requests,
- restart failed instances,
- scale replicas,
- perform rolling updates,
- allocate CPU/GPU resources.

**ML connection**

The model becomes a deployable service rather than simply a file on disk.

# Kubernetes and GPUs

ML workloads may require accelerators.
A Kubernetes cluster can contain nodes with:

- CPUs,
- GPUs,
- high-memory configurations,
- specialised accelerators.

A workload can request resources such as:

$$\text{CPU} + \text{Memory} + \text{GPU}$$

The orchestration platform can then schedule the workload onto a suitable node.

This allows a shared cluster to support:

$$\boxed{\text{Training} + \text{Inference} + \text{Other workloads}}$$

# What Is OpenShift?

**OpenShift** is an enterprise application platform built around Kubernetes.
It provides Kubernetes capabilities together with additional enterprise features and tooling.
Conceptually:

$$\boxed{\text{OpenShift} \approx \text{Kubernetes} + \text{Enterprise Platform Features}}$$

Typical capabilities include:

- Kubernetes orchestration,
- application deployment,
- security and access control,
- integrated developer workflows,
- container image management,
- monitoring and logging,
- enterprise administration.

# Kubernetes vs OpenShift

It is useful to think of them at different levels.

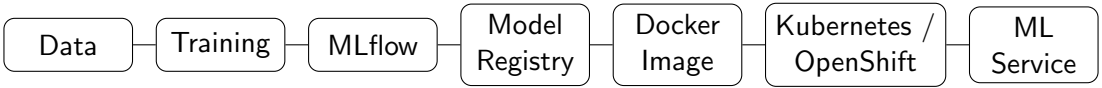
| | Kubernetes | OpenShift |
|---|---|---|
| Container orchestration | ✓ | ✓ |
| Workload scheduling | ✓ | ✓ |
| Scaling | ✓ | ✓ |
| Networking | ✓ | ✓ |
| Security tooling | Core capabilities | Additional enterprise tooling |
| Developer platform | Primarily Kubernetes | Integrated platform experience |
| Enterprise management | Depends on setup | Strong focus |

**Important**

OpenShift is not a replacement for the concepts learned in Kubernetes. It builds on Kubernetes and packages additional platform capabilities around it.

# Putting the Pieces Together

A possible MLOps architecture:




Each technology solves a different part of the problem.

# Who Does What?

| Technology | Main role | Example question |
|---|---|---|
| Git | Code versioning | What code produced this? |
| MLflow | ML lifecycle | Which experiment performed best? |
| Docker | Packaging | Can I reproduce this environment? |
| Kubernetes | Orchestration | Where/how should it run? |
| OpenShift | Enterprise platform | How do we operate this securely? |

Different tools solve different layers of the ML system

# From Training to Deployment

Consider a model that has been trained:

$$\text{Training} \rightarrow \text{Evaluation} \rightarrow \text{Model Registry}$$

If it passes the required evaluation:

$$\text{Model} \rightarrow \text{Docker Image}$$

Then:

$$\text{Docker Image} \rightarrow \text{Kubernetes / OpenShift}$$

Finally:

$$\text{Kubernetes} \rightarrow \text{Production API}$$

**This is the path from an experiment to a production service.**

# Continuous Integration and Deployment

MLOps often uses automation.
A simplified pipeline:

$$\boxed{\text{Git Commit} \rightarrow \text{Test} \rightarrow \text{Build Image} \rightarrow \text{Train / Evaluate} \rightarrow \text{Deploy}}$$

For example:

1. Developer changes inference code.
2. Automated tests run.
3. A new Docker image is built.
4. The image is stored in a registry.
5. Kubernetes/OpenShift deploys the new version.

This is the ML extension of familiar software engineering CI/CD practices.

# Continuous Training

Traditional software:

$$\text{Code change} \rightarrow \text{New release}$$

Machine learning adds another trigger:

$$\text{Data change} \rightarrow \text{Retraining}$$

A possible ML pipeline:

$$\text{New Data} \rightarrow \text{Validate} \rightarrow \text{Train} \rightarrow \text{Evaluate} \rightarrow \text{Register} \rightarrow \text{Deploy}$$

**Key idea**

In ML systems, the data itself can cause the software behaviour to change.

# Monitoring an ML System

Monitoring is not only about whether the server is running.
We may need to monitor:

**System**

- CPU
- GPU
- memory
- latency
- throughput
- errors

**ML**

- prediction quality
- data distributions
- feature drift
- prediction drift
- model performance

Healthy service $\neq$ Healthy ML system

# Model Drift

Suppose a model was trained on:

$$P_{\text{train}}(X)$$

But over time the production data becomes:

$$P_{\text{production}}(X)$$

If:

$$P_{\text{train}}(X) \neq P_{\text{production}}(X)$$

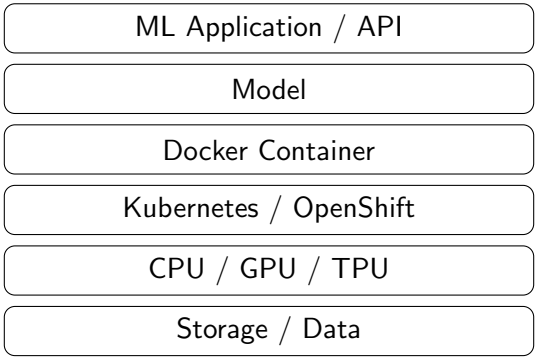
the model may perform differently.

Examples:

- customer behaviour changes,
- new products appear,
- sensors are replaced,

# The Infrastructure Stack

We can now view the ML system as layers:




MLOps connects these layers into a manageable lifecycle.

# A Student Project MLOps Architecture

Raw Data → Data Pipeline → Training

↓

MLflow → Best Model → Docker / Virtual Env → OpenShift/Kubernetes

↓

Scalable Inference API

The important principle is not that every project needs every tool.

**Design principle**

Choose infrastructure according to the requirements of the ML system.

# Common MLOps Failure Modes

- Training a model manually and losing the configuration.
- Using different preprocessing during training and inference.
- Deploying a model without recording its version.
- Building an application that only works on one machine.
- Running containers manually without orchestration.
- Not monitoring model performance after deployment.
- Retraining without knowing which data was used.
- Deploying a new model without a rollback strategy.

**MLOps goal**

Make these failure modes less likely through automation, versioning, testing and reproducibility.

# Key Takeaways

- MLOps applies software engineering and operations practices to machine learning systems.
- MLflow can track experiments and manage models.
- Docker packages an ML application and its dependencies.
- Kubernetes orchestrates containers and manages their lifecycle.
- OpenShift provides an enterprise platform built around Kubernetes.
- ML systems need monitoring in addition to ordinary infrastructure monitoring.
- Data changes can require models to be retrained.
- Versioning connects data, code, configuration and models.
- The goal is a repeatable path from experiment to production.